Instruction Format Specification – IA-32 (32-bit Only)

Corrected Assignment 1 Documentation based on the provided demo

Team Name: ia32-assembler

github link- https://github.com/purkararpita/ia32-assembler.git

25111067	Arpita Purkar	https://github.com/purkararpita
25111029	Kajal Kamble	https://github.com/kajalkamble321
25111042	Ajay pathade	https://github.com/ajay-collo
25111011	Vaibhav Dhale	https://github.com/Vaibhav724-web

1. Introduction

The Two-Pass Assembler is designed for the 32-bit x86 (IA-32) architecture and converts assembly language instructions into machine code. It supports registers, immediate values, memory addressing, ModR/M and SIB encoding, labels, jumps, and basic data directives. Instruction prefixes are not included in the scope of the assembler.

2. Assembly Syntax

General syntax:

[LABEL:] MNEMONIC [OPERAND1,[OPERAND2]]

- Label: Represents an instruction or data address.
- Mnemonic: Specifies the operation, e.g., MOV, ADD, SUB.
- Register: 32-bit general-purpose register such as EAX, EBX, ECX, etc.
- Immediate: Constant value, e.g., 10.
- Memory: Address expression, e.g., [EBX + 10h].
- Comment: Begins with ;.

3. Two-Pass Assembler

- Pass 1: Reads the source code, identifies labels, calculates instruction/data sizes, maintains the Location Counter, and creates the Symbol Table.
- Pass 2: Reads the source again, selects opcodes, generates ModR/M and SIB fields, resolves labels and offsets, and produces the final machine code.

Source Code → Pass 1 → Symbol Table → Pass 2 → Machine Code

4. Instruction Format

IA-32 instructions are variable-length and may contain the following fields. Not every instruction contains all fields.

Opcode	ModR/M	SIB	Displacement	Immediate
1–3 B	0/1 B	0/1 B	0/1/4 B	0/1/4 B

Instruction layout: Opcode → ModR/M → SIB → Displacement → Immediate

5. Register & ModR/M Encoding

The ModR/M byte contains MOD, REG/OPCODE and R/M fields. For MOD = 11, REG/OPCODE and R/M use the 32-bit register encoding table.

Register	Code
EAX	000
ECX	001
EDX	010
EBX	011
ESP	100
EBP	101
ESI	110
EDI	111

ModR/M format: 7 6 | 5 4 3 | 2 1 0 (MOD = 2 bits, REG/OPCODE = 3 bits, R/M = 3 bits)

- MOD = 00: Memory addressing
- MOD = 01: Memory + 8-bit displacement
- MOD = 10: Memory + 32-bit displacement
- MOD = 11: Register-to-register

Example: MOV EAX, EBX → MOD = 11, REG = 000, R/M = 011 → ModR/M = 11 000 011.

6. SIB & Addressing Modes

The SIB byte is used for scaled memory addressing.

SIB fields: SCALE (2 bits) | INDEX (3 bits) | BASE (3 bits)

- Register: EAX
- Immediate: 10
- Register indirect: [EBX]
- Base + displacement: [EBX + 10h]
- Base + index: [EBX + ESI]
- Base + index × scale: [EBX + ESI*4]
- Base + index × scale + displacement: [EBX + ESI*4 + 10h]

7. Instruction / Opcode Table (32-bit forms)

Instruction	Operand Form	Opcode
MOV	r/m32, r32	89 /r
MOV	r/m32, imm32	C7 /0
ADD	r/m32, r32	01 /r
ADD	r/m32, imm32	81 /0
SUB	r/m32, r32	29 /r
SUB	r/m32, imm32	81 /5
CMP	r/m32, r32	39 /r
CMP	r/m32, imm32	81 /7
XOR	r/m32, r32	31 /r
XOR	r/m32, imm32	81 /6
OR	r/m32, r32	09 /r
OR	r/m32, imm32	81 /1
AND	r/m32, r32	21 /r
AND	r/m32, imm32	81 /4
INC	r/m32	FF /0
DEC	r/m32	FF /1
MUL	r/m32	FF /4
DIV	r/m32	FF /6

8. Immediate, Displacement & Jumps

Immediate and displacement values may use 8-bit or 32-bit forms depending on the instruction and value. Multi-byte values use Little-Endian ordering.

Example: 12345678h → 78 56 34 12

For JMP and JNE, the resulting signed displacement is encoded with the selected jump form.

Displacement = Target Address − Address of Next Instruction

9. Directives & Symbol Table

Directive	Size	Example
DB	1 byte	DB 10
DW	2 bytes	DW 1234h
DD	4 bytes	DD 12345678h

Sections: SECTION .text and SECTION .data

.text contains instructions, while .data contains initialized data.

The Symbol Table stores labels and their addresses during Pass 1.

Symbol	Address	Type
START	00000000	Label
LOOP	00000010	Label

10. Sample Encoding

The assembly code will be converted into machine code using the following sequence:

Assembly Instruction

↓

Parse Operands

↓

Select Opcode

↓

Determine ModR/M

↓

Determine SIB

↓

Determine Displacement/Immediate

↓

Little-Endian Conversion

↓

Final Machine Code

The exact encoding will be performed using the sample code provided by the instructor.

11. Important Restriction Applied

- Only 32-bit general-purpose registers are included: EAX, ECX, EDX, EBX, ESP, EBP, ESI and EDI.
- 8-bit register/operand forms are not included in this documentation.
- 16-bit register/operand forms are not included in this documentation.
- The register operand forms used are r32 and r/m32.
- The immediate operand form used for 32-bit immediate values is imm32 (id in encoding notation).
- Memory operands use r/m32 and may use displacement and SIB addressing as required.

12. Conclusion

This corrected specification defines the required IA-32 syntax, instruction encoding, 32-bit register set, addressing modes, opcode selection, directives, and two-pass process for the proposed assembler. The documentation is based on the supplied assembler demo and is restricted to the 32-bit register and operand forms required for the project.